Sharded KV Store with Two-Phase Commit

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a distributed key-value store in which the data is partitioned across several servers, each server storing only a fraction of the keys, and a coordinator routes every client request to the right server (or set of servers). When a client writes a single key, the coordinator forwards the write to exactly one shard. When a client writes multiple keys that happen to land on different shards, the coordinator runs a *two-phase commit* protocol across all the involved shards, so that either every key is updated or none of them are, never a mixture. The assignment of keys to shards is done via consistent hashing, which is the standard technique for dividing a keyspace across machines in a way that survives the addition or removal of shards without massive data movement.

This project sits between the master-slave replication project (Option 1) and the CRDT project (Option 3) in its mix of concerns. Master-slave replication is about putting the same data on multiple machines for durability and read scaling; this project is about putting *different* data on different machines for storage and throughput scaling. CRDTs are about allowing conflicting concurrent writes to merge automatically; two-phase commit is about forcing multiple writes to commit atomically or not at all, which is the opposite philosophy. Understanding why both exist and why they solve fundamentally different problems is one of the points of the exercise.

The technical content breaks into three themes. First, consistent hashing: the algorithm used by Cassandra, DynamoDB, Memcached, and countless other distributed systems to map keys onto a ring of nodes in a way that tolerates node membership changes. Second, request routing: the coordinator needs to know, for each key, which shard owns it, and to forward the request. Third, two-phase commit: the classic distributed transaction protocol, which makes it possible to update multiple keys atomically across shards. You will implement all three from scratch, across four cooperating processes (one coordinator and three shards) communicating over TCP.

Two-phase commit is a famously imperfect protocol. Real distributed databases have moved toward more sophisticated alternatives (Paxos Commit, Percolator, Calvin, Spanner's TrueTime-based approach) that avoid 2PC's worst failure mode, the "blocking" problem when the coordinator crashes mid-commit. For this project, you will implement the basic 2PC protocol exactly as it is described in textbooks, observe its blocking behavior in a chaos test, and discuss the tradeoff in your design document. You are not expected to fix 2PC. You are expected to understand why it has the failure mode it has.

You are not implementing replication within a shard (each shard is a single process holding its data in memory with a simple WAL for durability), cross-shard reads as transactions (a multi-key `GET` reads the shards independently and the result may be inconsistent if a cross-shard write is in progress), or dynamic addition and removal of shards (the cluster has a fixed, hardcoded set of three shards, decided at startup). These are possible extensions in the bonus section.

2 What the Final Product Looks Like

When your project is complete, you can start one coordinator and three shard processes, connect a client to the coordinator, and watch it distribute writes across the shards and commit multi-key transactions atomically.

```
# terminal 1: shard 1
$ ./shardkv --shard 1 --port 7001 --data ./d1
[shard 1] WAL replay: 0 records
[shard 1] ready on port 7001

# terminal 2: shard 2
$ ./shardkv --shard 2 --port 7002 --data ./d2
[shard 2] ready on port 7002
```

```
# terminal 3: shard 3
$ ./shardkv --shard 3 --port 7003 --data ./d3
[shard 3] ready on port 7003

# terminal 4: coordinator
$ ./shardkv-coord --port 6001 \
                  --shards 1@h1:7001,2@h2:7002,3@h3:7003
[coord] ring built: 3 shards, 150 virtual nodes each (450 total)
[coord] connected to shard 1
[coord] connected to shard 2
[coord] connected to shard 3
[coord] ready on port 6001

# terminal 5: client
$ ./shardkv-cli h1 6001
connected to coordinator at h1:6001

> PUT alice 100
OK (shard 2)

> PUT bob 50
OK (shard 1)

> PUT carol 75
OK (shard 3)

> PUT dave 200
OK (shard 2)

> GET alice
100 (shard 2)

> \keys
alice  -> shard 2
bob    -> shard 1
carol  -> shard 3
dave   -> shard 2

# a multi-key transaction that spans two shards
> BEGIN
txn started (tx_id=1)

> PUT alice 90
staged on shard 2

> PUT bob 60
staged on shard 1

> COMMIT
PREPARE: sent to shards [1, 2]
PREPARE: shard 1 -> YES
PREPARE: shard 2 -> YES
DECISION: COMMIT
COMMIT: sent to shards [1, 2]
COMMIT: shard 1 -> ACK
COMMIT: shard 2 -> ACK
OK (tx_id=1 committed, 2 keys across 2 shards)

> GET alice
90 (shard 2)
> GET bob
60 (shard 1)

# what happens if a transaction fails
> BEGIN
```

```
txn started (tx_id=2)

> PUT xyz 42
staged on shard 3

> PUT alice 0
staged on shard 2

> FAIL  # user-simulated, same as if shard 3 had rejected PREPARE
aborting tx_id=2...
ABORT: sent to shards [2, 3]
ABORT: shard 2 -> ACK
ABORT: shard 3 -> ACK
(tx_id=2 aborted, no keys modified)

> GET alice
90 (shard 2)
# ^ alice is unchanged: the second transaction was aborted atomically.

> \stats
coordinator:     1
shards:          3
keys total:      4
keys per shard:  [bob] [alice,dave] [carol]
active txns:     0
committed txns:  1
aborted txns:    1
```

The defining moment of this session is the `COMMIT` output from the multi-key transaction: you see all four phases of two-phase commit printed on the console, in the order they happen. A working 2PC implementation shows `PREPARE` going out, all shards replying `YES`, the coordinator deciding, `COMMIT` going out, and all shards acknowledging. A broken one will hang, lose messages, or leave keys updated on some shards but not others. The chaos test in Phase 3 exercises the broken-implementation scenarios deliberately.

3 Deployment Options

The coordinator and shards are independent processes communicating over TCP. Deploy them however is most convenient.

3.1 Four Processes on One Machine

Open four terminals and start the coordinator and three shards on different ports, using `localhost` in the shard list. This is the recommended development setup by default. `tmux` with four panes makes it easy to watch all four logs simultaneously.

3.2 Docker Compose

Four services in a `docker-compose.yml`: three shards and one coordinator. `docker stop shard1` is the cleanest way to simulate a shard failure for chaos testing.

3.3 Separate Machines

Run the coordinator on one machine and the shards on three others. This is realistic but the most tedious to set up; only choose it if you have a specific reason to test over a real network.

3.4 Fixed Cluster Size

The cluster has exactly one coordinator and exactly three shards. The coordinator is a single point of failure — if it crashes, the cluster is unavailable until it is restarted. Production 2PC systems usually replicate the coordinator itself (often with Raft) to avoid this, but doing so is out of scope. In your chaos test you will deliberately kill the coordinator and observe the blocking behavior.

4 Consistent Hashing

Consistent hashing is the algorithm that decides, for a given key, which shard owns it. It is the single idea that makes this project distinct from the other distributed projects in Project 02, and it is worth understanding deeply before writing any code.

4.1 The Naive Approach and Why It Fails

The simplest way to distribute keys across shards is modulo arithmetic: $shard(k) = hash(k) \bmod N$, where N is the number of shards. This works, but it has a catastrophic property: if N changes, almost every key moves. Go from three shards to four and roughly three-quarters of all keys have to be relocated. Go from four to three and the same thing. In a real deployment where nodes are added or removed from a running cluster, this is unacceptable.

4.2 The Ring

Consistent hashing fixes this by imagining the hash space as a circle. The output of the hash function is treated as a point on the ring: the range $[0, 2^{32})$ wraps around, so position $2^{32} - 1$ is immediately followed by position 0. Every shard is assigned a position on the ring (more on how below). Every key is also assigned a position on the ring by hashing it. The shard that owns a key is the first shard encountered when walking clockwise from the key's position.

```
                    0
                    |
                shard1
                  / | \
                 /  |  \
                /   |   \
        key3-o      |    o-key1  -> owned by shard 2
            /       |     \
           /        |      \
        shard3----+----shard2
           \        |      /
            \       |     /
             \      |    /
              \     |   /
               \    |  /
            key2-o  |   -> owned by shard 3
                \ |
                 \|
            (wraparound)
```

When a shard is added, it is assigned a new position on the ring. Only the keys that fall between the new shard's position and the previous clockwise shard need to move. When a shard is removed, the keys it used to own are now owned by the next clockwise shard, and only those keys move. In both cases the number of keys that move is approximately $1/N$ of the total, which is the best you can hope for.

4.3 Virtual Nodes

There is one problem with the simple ring. If each shard has exactly one position on the ring, the ring is divided into N unequal arcs, and the distribution of keys to shards is uneven. With three shards assigned random positions, one shard might end up with 60% of the ring and another with 10%. The fix is called *virtual nodes*: each physical shard is assigned many positions on the ring (for example, 150) instead of just one. With 150 virtual positions per shard and 3 shards, the ring has 450 positions, and each arc is small enough that, by the law of large numbers, each shard ends up with roughly $1/N$ of the total keys.

In code, this is implemented by computing, for each physical shard, 150 distinct hash values — typically `hash("shard1-0")`, `hash("shard1-1")`, ..., `hash("shard1-149")` — and inserting all 150 into the ring. When looking up the owner of a key, the ring is searched for the first virtual position $\geq hash(k)$, and the shard that owns that virtual position is the answer. The virtual positions are normally stored in a sorted array so the lookup is a binary search.

4.4 The Hash Function

Use a simple, deterministic, non-cryptographic hash. The FNV-1a hash (Fowler-Noll-Vo) is a good choice: it is a few lines of C, it produces a uniform distribution for typical inputs, and it is fast. MurmurHash3 is another common choice if you prefer. Do not use `rand()` or any hash that produces different outputs on different invocations. The hash function must be pure: the same input always

produces the same output.

```
uint32_t fnv1a(const char *data, size_t len) {
    uint32_t h = 2166136261u;
    for (size_t i = 0; i < len; i++) {
        h ^= (unsigned char) data[i];
        h *= 16777619u;
    }
    return h;
}
```

4.5 Ring Construction and Lookup

At startup, the coordinator builds the ring from its configured shard list. Construction is:

1. For each shard s in the configured list, and for each virtual node index i in $[0, 149]$:
   - Compute $p = fnv1a($`"shard_name-" + i`$)$.
   - Insert (p, s) into the ring.
2. Sort the ring by p, producing a sorted array of $(position, shard)$ pairs.

Lookup for a key k is:

1. Compute $h = fnv1a(k)$.
2. Binary search the ring for the smallest $p \geq h$. If none exists (because h is larger than every position in the ring), wrap around and use the first position in the array. This handles the clockwise wraparound.
3. Return the shard associated with that position.

The binary search is $O(\log(N \times V))$ where V is the virtual node count. For three shards and 150 virtual nodes that is about 9 comparisons per key lookup — fast enough that lookup will never be your bottleneck.

4.6 Testing the Ring

Write a unit test that verifies key distribution is approximately even. Generate 100,000 random keys, look up the owner of each, and count how many land on each shard. With 150 virtual nodes, each shard should receive between 28% and 38% of the keys. Significant deviations indicate a bug in your hash function or in the ring construction.

A separate test verifies the "addition doesn't move many keys" property. Build a ring with two shards and look up 10,000 keys; remember the owner of each. Then rebuild the ring with three shards and look up the same 10,000 keys. Count how many changed owners. The expected fraction is about 1/3 (the keys that now belong to the new shard); anything dramatically higher means your ring construction is not deterministic, probably because you did not actually sort it.

5 Two-Phase Commit

Two-phase commit is the protocol the coordinator uses to atomically update a set of keys that live on different shards. It is the second major idea in this project, and like consistent hashing, its details are worth getting right.

5.1 The Problem

Suppose a client wants to transfer 100 units from a balance stored on shard 1 to a balance stored on shard 2: `PUT alice balance_minus_100` on shard 1 and `PUT bob balance_plus_100` on shard 2. If the coordinator simply sends each update independently, three bad things can happen:

- The first update succeeds, but the second fails (shard 2 is unreachable, or rejects the write for some reason). Money has been destroyed.
- The first update succeeds, the second fails, the coordinator retries the second successfully. Now suppose between these two steps, a concurrent reader asked shard 1 and shard 2 for their balances. It would see a total that is 100 units short, because shard 1 has decremented but shard 2 has not yet incremented.

- The first update succeeds, the coordinator crashes before sending the second. On restart, the coordinator has no memory of the half-finished operation, and the cluster is in an inconsistent state that no one can detect.

Two-phase commit prevents all three by having every participating shard first *promise* to be able to perform its update, then having the coordinator collect all promises and make a single atomic decision, then having all shards either perform the update or discard it depending on the decision.

5.2 The Protocol

A transaction in 2PC has these actors: a single coordinator, and a set of participants (shards that will be asked to do work). Every participant has two logs: a durable log that survives crashes, and an in-memory "staged" state that holds the transaction's pending modifications.

The protocol runs in four phases:

1. **Staging.** The client sends each individual `PUT` or `DELETE` to the coordinator within the scope of a `BEGIN`...`COMMIT` block. For each operation, the coordinator looks up the owning shard and sends a `STAGE` message to that shard. The shard records the operation in its in-memory staging area (indexed by transaction id) but does not yet apply it to its main data. It replies `OK`. At this stage the coordinator just collects which shards are involved in the transaction.
2. **Prepare.** When the client issues `COMMIT`, the coordinator sends a `PREPARE` message to each participating shard. Each shard, upon receiving `PREPARE`, decides whether it is willing to commit: it writes a `PREPARE` record to its durable WAL (including the staged operations), then replies `YES` ("I am ready and promise to commit if asked") or `NO` ("I refuse to commit," with a reason). After sending `YES`, the shard has given up its right to unilaterally abort — it must commit if told to, even if it crashes and restarts, because its durable WAL says so.
3. **Decision.** The coordinator collects the responses. If *all* shards replied `YES`, the coordinator's decision is `COMMIT`. If any shard replied `NO`, or if any shard timed out, the decision is `ABORT`. The coordinator writes the decision to its own durable WAL before acting on it. Once this decision record is durable, the fate of the transaction is fixed; the coordinator cannot change its mind.
4. **Commit or Abort.** The coordinator sends the decision (`COMMIT` or `ABORT`) to every shard that was in the transaction. Each shard, upon receiving it: if the decision is `COMMIT`, applies the staged operations to its main data and writes a `COMMIT` record to its WAL; if the decision is `ABORT`, discards the staged operations and writes an `ABORT` record. Either way, it replies `ACK` to the coordinator. The coordinator collects all acknowledgements and then reports success (or failure) to the client.

5.3 Why This Works (and When It Doesn't)

The protocol achieves atomicity because the coordinator's decision is written durably before any commit message is sent. If the coordinator crashes after writing the decision, on restart it can read the decision and resend the `COMMIT` or `ABORT` messages to every shard. Shards that already received the decision acknowledge again (they are idempotent). Shards that did not receive it the first time receive it now.

Similarly, the participants' durable `PREPARE` records mean that if a shard crashes after replying `YES` but before receiving the decision, on restart it remembers it is "in doubt" about transaction t: it has promised to commit t if told to, but does not know what the coordinator decided. The shard will wait (or, in a production system, ask the coordinator; for this project, the shard can simply wait until the coordinator reconnects and re-sends the decision).

The protocol **does not** handle the case where the coordinator crashes before sending the decision and never recovers. In that case the shards that have voted `YES` are stuck: they cannot abort (because they have given up their right to unilaterally abort), and they cannot commit (because they do not know the decision). They wait forever. This is the famous "blocking" failure mode of 2PC, and it is a genuine weakness of the protocol that more sophisticated variants (three-phase commit, Paxos Commit) attempt to fix. In this project you will observe the blocking behavior in your chaos test and describe it in the design document.

5.4 Single-Key Transactions Are a Special Case

A client write that touches only one key does not need to go through 2PC. The coordinator can simply forward the `PUT` to the single owning shard, which applies it and WAL-logs it immediately. This is the common case and should be fast. Only multi-key transactions, wrapped in `BEGIN`...`COMMIT`, use the full protocol.

A subtle case: a multi-key transaction whose keys all happen to land on the *same* shard. The coordinator should notice this and skip the 2PC dance entirely, sending the set of operations as a single atomic batch to the one shard. Your implementation should handle this optimization or document why it does not.

6 The Protocols

There are two protocols in this project: the client-to-coordinator protocol (plain text, line-oriented, simple) and the coordinator-to-shard protocol (binary, message-framed, used for everything 2PC-related as well as single-key operations).

6.1 Client-to-Coordinator Protocol

Line-based plain text, terminated by newlines.

- `PUT key value` — Single-key put.
- `GET key` — Single-key get. Always routed to the owning shard.
- `DELETE key` — Single-key delete.
- `BEGIN` — Start a multi-key transaction. The coordinator responds with a `tx_id`.
- `COMMIT` — End the current transaction with the intent to commit. Triggers the 2PC protocol.
- `ABORT` — End the current transaction discarding its changes.
- `\keys` — List all known keys and which shard owns each (coordinator-side knowledge only; if shards have keys the coordinator never wrote, they will not appear here).
- `\stats` — Print cluster statistics.
- `QUIT`.

6.2 Coordinator-to-Shard Protocol

Binary, message-framed. Every message starts with a 12-byte header:

```
Offset  Size   Field           Description
------  -----  --------------  -------------------------------------
  0      4     msg_len         uint32, total length including header
  4      2     msg_type        uint16 (see below)
  6      2     flags           uint16, currently unused (must be 0)
  8      4     tx_id           uint32, the transaction id
                               (0 for single-key operations)
 12     ...    body            type-specific
```

Message types:

- `PUT_ONE (1)`: single-key put. Body: 4-byte key length, key, 4-byte value length, value. Shard replies with `PUT_ONE_RESP (2)` containing a status byte (1 = OK, 0 = error) and, on error, a message.
- `GET_ONE (3)`: single-key get. Body: key. Shard replies with `GET_ONE_RESP (4)`: status byte, then value length and value if OK.
- `DEL_ONE (5)`: single-key delete. Body and reply mirror `PUT_ONE`.
- `STAGE (10)`: stage an operation inside a transaction. Body: op type (1=PUT, 2=DELETE), key, and (for PUT) value. Shard replies with `STAGE_RESP (11)`: OK or error.
- `PREPARE (20)`: begin the prepare phase. Body is empty. Shard replies with `PREPARE_RESP (21)`: vote byte (1=YES, 0=NO) and, on NO, a reason string.
- `COMMIT (30)`: apply the staged operations. Body is empty. Shard replies with `COMMIT_RESP (31)`: ack byte.
- `ABORT (40)`: discard the staged operations. Body is empty. Shard replies with `ABORT_RESP (41)`: ack byte.

Every single message is framed by the 12-byte header's `msg_len`, which is the *total* length of the message

including the header itself. The receiver reads exactly 12 bytes for the header, extracts `msg_len`, then reads exactly `msg_len - 12` more bytes for the body. Partial reads are possible (TCP does not guarantee message boundaries), so the receive loop must accumulate bytes until the full message is present.

7 The Shard

Each shard is a small TCP server that holds its own independent key-value store. It accepts connections from the coordinator (it does not serve clients directly) and handles the binary protocol above.

7.1 Data Structures

- A main key-value store: a hash map from key string to value string, holding all committed data.
- A staging area: a hash map from `tx_id` to a list of staged operations. An operation is `(op_type, key, value)`.
- A WAL (write-ahead log) for durability, same format as in earlier projects: append-only file, CRC-checked records, `fsync` on every commit. Each record is a binary structure with a type, a `tx_id`, and the operation data.

7.2 Handling Each Message Type

PUT_ONE / DEL_ONE / GET_ONE. Single-key operations. Look up the key in the main store; for writes, append a `SINGLE` record to the WAL, `fsync`, then apply to the main store and reply.

STAGE. Record the operation in the staging area under the given `tx_id`. Nothing is logged yet. Reply OK.

PREPARE. Look up the staging area entry for this `tx_id`. Decide whether you are willing to commit (for this project, the answer is always YES unless the staging area is empty, which would be a protocol bug). Write a `PREPARE` record containing the full list of staged operations to the WAL and `fsync`. Reply YES.

COMMIT. Read the staged operations for this `tx_id`, apply them to the main store, write a `COMMIT` record to the WAL and `fsync`, remove the staging entry. Reply ACK.

ABORT. Write an `ABORT` record to the WAL and `fsync`, remove the staging entry. Reply ACK.

7.3 Crash Recovery

On startup, the shard replays its WAL to rebuild state. The rules are:

- A `SINGLE` record: apply the operation to the main store.
- A `PREPARE` record with no corresponding `COMMIT` or `ABORT`: this transaction is *in doubt*. Rebuild the staging entry from the PREPARE record's operation list. The shard must not apply the operations; it must wait for the coordinator to reconnect and tell it the fate of the transaction.
- A `PREPARE` followed by a `COMMIT`: apply the operations (as if the commit just arrived) and discard the staging entry.
- A `PREPARE` followed by an `ABORT`: discard the staging entry without applying.

The "in doubt" case is the one that demonstrates 2PC's blocking property. A shard in this state cannot make progress on the affected keys until the coordinator is available again. It can still serve single-key operations on other keys; the freeze is per-transaction, not global.

8 The Coordinator

The coordinator is a more substantial process than any single shard. It holds the ring, routes client requests, and runs the 2PC protocol for multi-key transactions.

8.1 Data Structures

- The consistent hash ring, built at startup from the configured shard list.
- A map from shard id to an open TCP connection to that shard.
- A transaction table: a hash map from `tx_id` (locally assigned by the coordinator, monotonic) to a `Transaction` struct containing the client connection that owns it, the list of staged operations (used for routing), and the set of involved shards (computed as staging happens).
- A counter for the next `tx_id`.

- A durable WAL for coordinator-side crash recovery. Record types: `TX_BEGIN`, `TX_PREPARE_COMPLETE` (listing which shards voted YES), `TX_COMMIT_DECISION`, `TX_ABORT_DECISION`, `TX_DONE`.

8.2 Request Handling

Single-key operations (`PUT`, `GET`, `DELETE` outside a transaction): look up the key's owning shard via the ring, forward to that shard as a `PUT_ONE`/`GET_ONE`/`DEL_ONE`, wait for the response, forward the response to the client. Log nothing to the coordinator's WAL; single-key operations are the shard's responsibility to durably log.

`BEGIN`: allocate a new `tx_id`, create a `Transaction` struct, respond to the client with the id. Log a `TX_BEGIN` record.

`PUT/DELETE` **inside a transaction**: look up the owning shard, add the shard to the transaction's involved-shard set, send a `STAGE` message to that shard with the `tx_id` and the operation, wait for OK, respond to the client with "staged on shard s."

`COMMIT`: run the 2PC protocol:

1. If the involved-shard set has exactly one shard, short-circuit: send a `COMMIT` directly to that shard and skip the 2PC dance. Log nothing extra.
2. Otherwise, send `PREPARE` to every involved shard in parallel.
3. Collect responses. If all return YES within a timeout (e.g., 5 seconds), log a `TX_COMMIT_DECISION` record and `fsync`. Then send `COMMIT` to every involved shard, collect ACKs, respond to the client. Finally log a `TX_DONE` record (this is not strictly necessary for correctness but is useful for WAL cleanup).
4. If any shard returned NO, or if any shard timed out, log a `TX_ABORT_DECISION` record, `fsync`, send `ABORT` to every involved shard, respond to the client with an error.

`ABORT` (client-initiated): if no `PREPARE` has been sent yet, simply discard the transaction locally. If `PREPARE` has been sent, the coordinator is already committed to a decision; this case should not arise if the client protocol is followed, but if it does, the coordinator can either refuse or send `ABORT` to all involved shards.

8.3 Coordinator Crash Recovery

On startup, the coordinator replays its WAL and recovers incomplete transactions:

- For any `tx_id` with a `TX_COMMIT_DECISION` but no `TX_DONE`: resend `COMMIT` to every involved shard. (The shards are idempotent about duplicate commits: a shard that already committed simply replies ACK again.)
- For any `tx_id` with a `TX_ABORT_DECISION` but no `TX_DONE`: resend `ABORT` to every involved shard.
- For any `tx_id` with a `TX_BEGIN` but no decision: abort it. The client that started the transaction is no longer connected, so the correct thing to do is send `ABORT` to any shards that saw `STAGE` or `PREPARE` messages. Since the coordinator may not know which shards those were (the `STAGE` records are per-operation and not durably logged in the base project), a safe approach is to broadcast `ABORT` for this `tx_id` to every shard; shards without any record of the transaction simply reply ACK and do nothing.

The third bullet highlights a design choice: the base project does not durably log every `STAGE` on the coordinator side, which means recovery is slightly imprecise (abort-broadcast to all shards). A more precise implementation would log each `STAGE` decision as well. Either approach is acceptable; document whichever you picked.

9 Phase 1 — The Shard and Consistent Hashing

The goal of Phase 1 is a working shard process and a coordinator that routes single-key operations. No transactions, no 2PC. A client that asks to `PUT` or `GET` a single key should be able to do so, and the coordinator should use the ring to pick the right shard for each key.

Build these components:

- The shard process (`shardkv`) with the hash map store, the WAL writer, the WAL replay, and the message handlers for `PUT_ONE`, `GET_ONE`, and `DEL_ONE`.
- The consistent hash ring (Section 4) as a library used by the coordinator. Include the unit tests described in Section 4.6: distribution uniformity and minimal key movement on resize.
- The coordinator process (`shardkv-coord`) with the ring, TCP connections to the configured shards (opened at startup), and handlers for single-key client commands.
- The client (`shardkv-cli`).
- A simple `\keys` command that is built up lazily: every time the coordinator routes a write, it remembers which shard it went to, and `\keys` dumps this map.

At the end of Phase 1, a three-shard cluster should handle single-key operations correctly, with the ring distributing keys approximately evenly. Kill a shard and the coordinator should report errors for the keys that happen to live on that shard, while still serving the rest.

10 Phase 2 — Two-Phase Commit

The goal of Phase 2 is multi-key transactions via 2PC.

Build these components:

- On the shard side, the staging area, the `STAGE`/`PREPARE`/`COMMIT`/`ABORT` message handlers, and the WAL records for `PREPARE`, `COMMIT`, and `ABORT`. Update WAL replay to reconstruct in-doubt staged transactions.
- On the coordinator side, the transaction table, the `tx_id` allocator, the `BEGIN`/`COMMIT`/`ABORT` client handlers, the 2PC coordinator logic (prepare phase, decision, commit phase), and the coordinator WAL with `TX_BEGIN`, `TX_COMMIT_DECISION`, `TX_ABORT_DECISION`, `TX_DONE`.
- Coordinator crash recovery logic that resends decisions for in-flight transactions on startup.
- The single-shard short-circuit optimization (Section 5.4).

At the end of Phase 2, the example session in Section 2 should work end to end. A multi-key transaction spanning two shards should commit atomically: either both keys reflect the new values, or neither does.

10.1 Correctness Testing

Write a test that exercises 2PC's core promises:

1. Start the cluster. Do 100 multi-key transactions, each touching 2 shards. Verify that every committed transaction is visible on every shard it touched, and no aborted transaction's effects are visible anywhere.
2. Repeat with transactions that touch 3 shards.
3. Do a transaction that deliberately triggers an abort (for example, simulated by adding a command-line flag or a shell environment variable that makes one specific shard reply NO to `PREPARE`). Verify the transaction does not commit on any shard.

11 Phase 3 — Chaos Testing and Benchmark

The goal of Phase 3 is to stress-test the 2PC implementation against failures and to produce a benchmark.

11.1 Chaos Scenarios

Write a chaos test driver that runs the following scenarios and verifies the invariants after each. Every scenario must either pass or be documented (in the design document) as a known failure mode of 2PC.

1. **Shard crash during PREPARE.** Start a multi-key transaction. In the middle of the prepare phase, after one shard has received `PREPARE` but before the second has, kill the second shard's process. Verify that the coordinator aborts the transaction. Restart the shard. Verify that its WAL replay does not leave any stale staged transactions.
2. **Shard crash after PREPARE but before COMMIT.** Start a multi-key transaction. Let all shards complete `PREPARE` with YES votes. Let the coordinator write its `TX_COMMIT_DECISION`. Kill one of the participating shards before it receives `COMMIT`. Verify that the transaction's effects are visible on the other shards. Restart the killed shard. Verify that its WAL replay leaves the transaction

in the "in doubt" state, that the coordinator detects the in-doubt state on reconnect and resends `COMMIT`, and that the shard then finalizes.

3. **Coordinator crash between PREPARE and decision.** Start a multi-key transaction. Let all shards receive `PREPARE` and reply YES. Kill the coordinator before it writes the decision record. Observe that the shards are now in the "in doubt" state. Verify that single-key operations on keys *not* involved in the in-doubt transaction continue to work. Restart the coordinator. Because the coordinator has no decision record, its recovery logic aborts the transaction, sending `ABORT` to both shards, which discard their staged operations. **This is the blocking failure mode of 2PC.** Document it.
4. **Coordinator crash after writing COMMIT decision.** Start a multi-key transaction. Let all shards reply YES, let the coordinator write its `TX_COMMIT_DECISION`, and kill the coordinator before it can send `COMMIT` to any shard. The shards are in doubt but the decision exists on disk. Restart the coordinator. Verify that recovery reads the decision, resends `COMMIT`, and the transaction completes correctly.
5. **Network partition between coordinator and one shard.** Start a multi-key transaction touching three shards. Arrange for one shard to be unreachable from the coordinator at the moment `PREPARE` is sent. Verify that the coordinator times out waiting for that shard's response and sends `ABORT` to the other shards. The other shards must discard their staged operations.
6. **Concurrent single-key writes during 2PC.** Start a multi-key transaction. While it is in the middle of 2PC, have another client issue single-key writes to keys that are *not* involved in the transaction. Verify that those writes succeed normally and are visible immediately. This verifies that 2PC does not lock the entire shard.

The chaos test driver is the most important deliverable of Phase 3. The design document must describe the observed behavior for each scenario, with particular attention to the blocking case in scenario 3.

11.2 Benchmark

Measure the throughput of the cluster for the three common operation types:

- Single-key puts: fire 10,000 independent single-key `PUT`s through the coordinator (keys distributed approximately evenly across shards). Report puts per second.
- Multi-key 2PC transactions: fire 1,000 multi-key transactions, each touching two randomly chosen shards with two keys. Report transactions per second.
- Single-shard multi-key transactions: fire 1,000 multi-key transactions whose keys all happen to land on the same shard (this exercises the short-circuit optimization from Section 5.4). Report transactions per second, and verify it is significantly higher than the two-shard case.

Include the numbers in the design document with a brief analysis. Expect 2PC to be noticeably slower than single-key operations because every commit involves two round trips to every participating shard. This is the fundamental cost of distributed transactions.

12 Deliverables and Submission

12.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation, including the shard server (`shardkv`), the coordinator (`shardkv-coord`), the client (`shardkv-cli`), and the chaos test driver.
- A `Makefile` (or `CMakeLists.txt`) that builds all binaries with a single command on a recent Ubuntu LTS.
- A `README.md` with build instructions, cluster startup instructions for each deployment option in Section 3, the list of supported commands, and any known limitations.
- A `design.pdf` document describing: the consistent hash ring implementation and the virtual node design; the wire format for both protocols; the shard's data structures and WAL record types; the coordinator's transaction table and WAL record types; the exact message flow of a 2PC commit, with a sequence diagram or equivalent; the crash recovery logic on both sides; a detailed walkthrough of each chaos scenario including the observed behavior; and an explicit discussion of the blocking failure

mode in scenario 3.
- A `chaos/` directory with the chaos test driver and the logs from your final successful run.
- A `docker-compose.yml` or equivalent for whichever deployment option you tested with.
- A `tests/` directory with unit tests for the consistent hash ring (distribution uniformity, minimal movement on resize), the message framing (serialize and deserialize round trip), and the shard's WAL replay logic (hand-crafted log files producing expected states).

12.2 Archive Naming

`Group[Number]_Project02_Sharding.zip`, for example `Group17_Project02_Sharding.zip`.

12.3 Demonstration

Each group will present a live demonstration during the week following the submission evaluation. It consists of three parts. First, a brief walkthrough of the architecture using the design document, with particular attention to the ring and the 2PC message flow. Second, a live execution of the chaos test, with the instructor/TAs free to propose additional crash scenarios on the spot ("what happens if I kill the coordinator exactly after one shard has replied YES but before the other?"). Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component of the implementation, with particular focus on why the coordinator logs its decision *before* sending commit messages and what would go wrong if it logged after. All group members must be present. Missing the demonstration results in a zero for Phase 3.

13 Bonus Opportunities

Each of the following is independent. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points. To claim bonus credit, your design document must include a dedicated section explaining which items you attempted and how you verified them.

13.1 Dynamic Shard Addition (up to 10%)

The base project has a fixed set of three shards chosen at coordinator startup. Implement dynamic addition: a new shard joins a running cluster, the coordinator learns of it (via a configuration reload command), the ring is rebuilt with virtual nodes for the new shard, and the keys that now belong to the new shard are migrated from their old owners to it. Migration must happen without dropping requests: during the move, reads and writes for those keys may be slightly slower, but must remain correct. Describe your migration protocol in the design document and demonstrate it with a test that adds a fourth shard to a running cluster and verifies no data is lost.

13.2 Parallel Prepare (up to 3%)

The base project sends `PREPARE` to each shard sequentially (the description says "in parallel" but a simple implementation is often sequential). Implement true parallel prepare: send the messages to all shards concurrently, collect responses as they arrive, and make the decision as soon as either the first NO arrives (abort early) or the last YES arrives (commit early). Measure the improvement in 2PC latency compared to a sequential baseline.

13.3 Read Isolation (up to 5%)

The base project allows a `GET` to see a half-committed transaction (for example, a cross-shard transfer where shard 1 has committed but shard 2 has not). Implement snapshot isolation: each shard tags every committed value with its transaction id, and a `GET` for a key inside a `BEGIN...COMMIT` block reads a consistent snapshot corresponding to the transaction's start time. This requires the shard to keep multiple versions of each key. Describe the mechanism and demonstrate it with a test that performs a transaction whose reads see a consistent view even while concurrent writes are happening.

14 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing distributed key-value store (Cassandra, Riak, Voldemort, FoundationDB, TiKV, CockroachDB, Redis Cluster, etc.) as a component of your implementation.
- Use of any existing consistent hashing library. The ring in Section 4 must be your own code.
- Use of any existing transaction coordinator or two-phase commit library. The 2PC protocol must

be implemented from scratch.

- Use of any existing binary serialization library (Protobuf, Cap'n Proto, FlatBuffers, MessagePack). The binary message format in Section 6.2 must be implemented with direct reads and writes on a plain TCP socket.
- Skipping `fsync` on WAL writes. Both the shards and the coordinator must `fsync` their WALs at the points specified in Sections 7 and 8; skipping this breaks the durability argument for 2PC and will manifest as inconsistency after crash-recovery scenarios.
- Using modulo hashing (`hash(k) % N`) instead of consistent hashing with virtual nodes. The ring is a hard requirement.
- Coordinating across shards without using 2PC for multi-key writes. The protocol must follow the PREPARE / decision / COMMIT structure described in Section 5, with durable logging at the specified points.
- Use of Python at any layer, including test harnesses and chaos test drivers. Tests and scripts must be written in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits. Your git history should reflect the incremental development of a system of this size.

15 Required Reading

The first two items are required reading before you start Phases 1 and 2 respectively. The remaining items are reference material.

1. Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., and Lewin, D. "Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web." *Proceedings of the ACM Symposium on Theory of Computing*, 1997. The original consistent hashing paper. Short and readable; Section 4 is the one that matters for this project. Read it before Phase 1.
2. Bernstein, P. A., Hadzilacos, V., and Goodman, N. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. Chapter 7 on atomic commitment is the classic reference for 2PC. It is long and formal; the important parts are the protocol description (Section 7.2) and the discussion of failure modes (Section 7.4). Read those before Phase 2. The book is free online at the author's website.
3. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 9 ("Consistency and Consensus"), specifically the section on atomic commit and two-phase commit. A more approachable modern discussion of 2PC, including the blocking failure mode and alternatives.
4. Lamport, L., and Massa, M. "Cheap Paxos." *Proceedings of the International Conference on Dependable Systems and Networks*, 2004. Optional. Introduces the idea of replicating the coordinator itself with Paxos, which is one of the standard fixes for 2PC's blocking failure. Relevant if you are curious about production systems.
5. DeCandia, G. et al. "Dynamo: Amazon's Highly Available Key-Value Store." *ACM Symposium on Operating Systems Principles*, 2007. Section 4.2 on partitioning describes exactly the consistent hashing with virtual nodes that you will implement. Useful as a reference for how a production system organizes the ring.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM